Exercise 2

Basic Arithmetic and Status Flags

Objectives

- Implementation of a digital 8 bit ripple carry adder and subtractor circuit
- Generation of status flags
- Test the functionality on an FPGA (Device XC7A100T CSG324-1)
- Evaluate the propagation delay

General Description

In this lab exercise a simple arithmetic unit for addition and subtraction has to be implemented in VHDL and mapped onto an FPGA. To this end, a full adder cell (FA) has to be described in VHDL and scaled up to 8 bit. Moreover, the status flags for overflows (signed (V) and unsigned (C)), zero (Z) and negative (N) values of the calculation have to be implemented. The resulting VHDL description has to be embedded into the Vivado Design Tool and validated by meaningful test stimuli. Next, logical and physical synthesis has to be carried out and analyzed in terms of complexity and timing. The resulting bitstream is mapped onto an Artix-7 FPGA (XC7A100T) and tested, using the HAW MODSYS evaluation board with IOMOD extension boards (see Fig. 1).

After the session, each group has to write and submit a lab report (see `General Advises and Rules` in TEAMs) as pdf via TEAMs no later than one week after the session: Deadline: Thursday 11.59 pm (hard deadline, no extensions possible). If no report has been submitted, this will result in a score of 0 points for the exercise and team.

MOSYS evaluation board and Extension boards

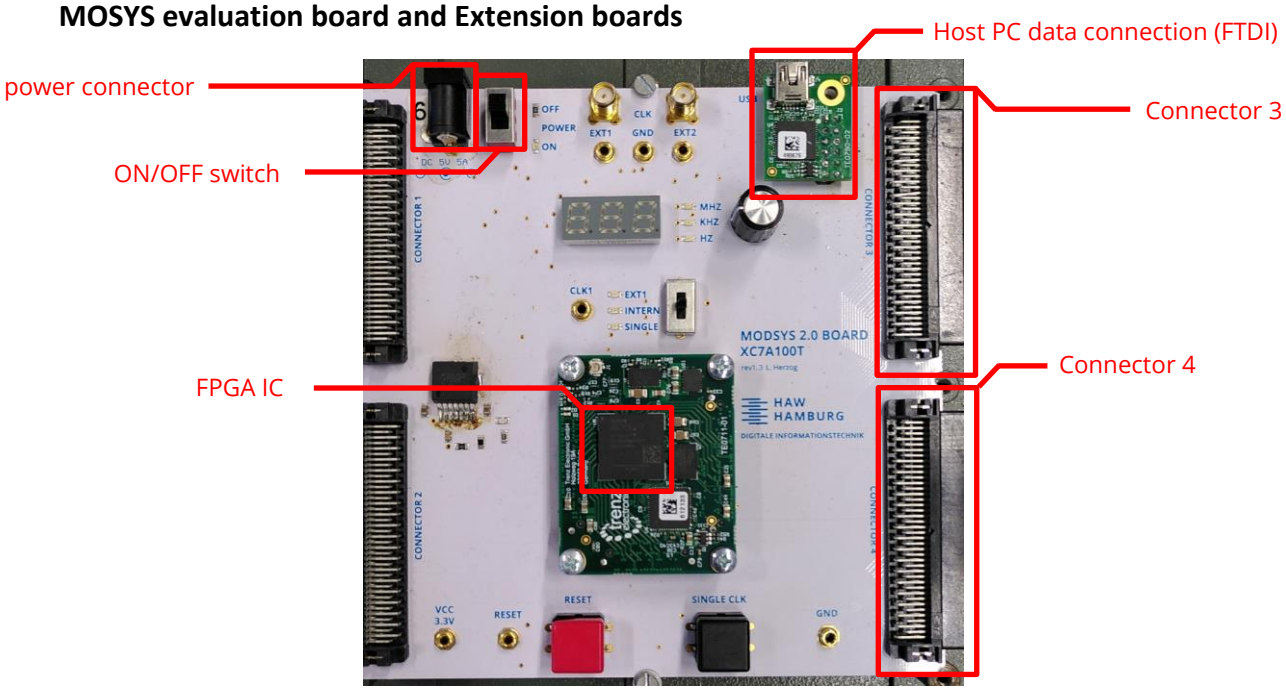


Fig. 1: MODSYS 2.0 Evaluation Board

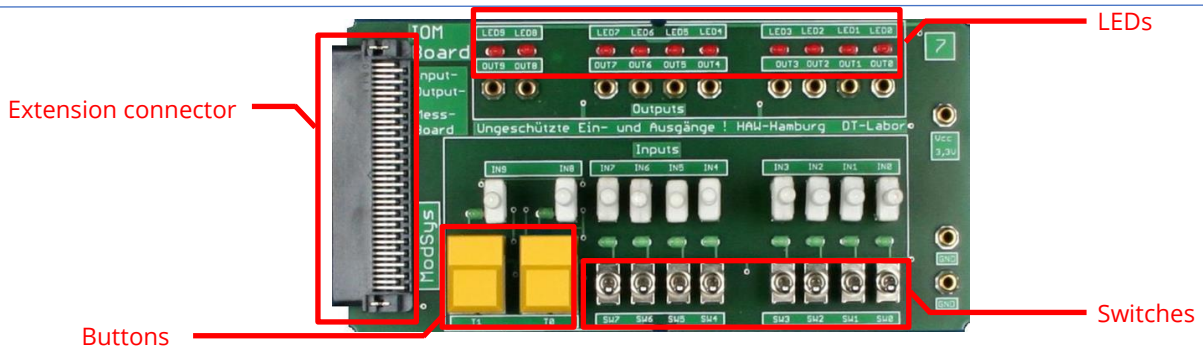


Fig. 2: IO-Module extension card

For the exercise, the two IO extension cards are connected to the MODSYS 2.0 as shown in Fig 2.

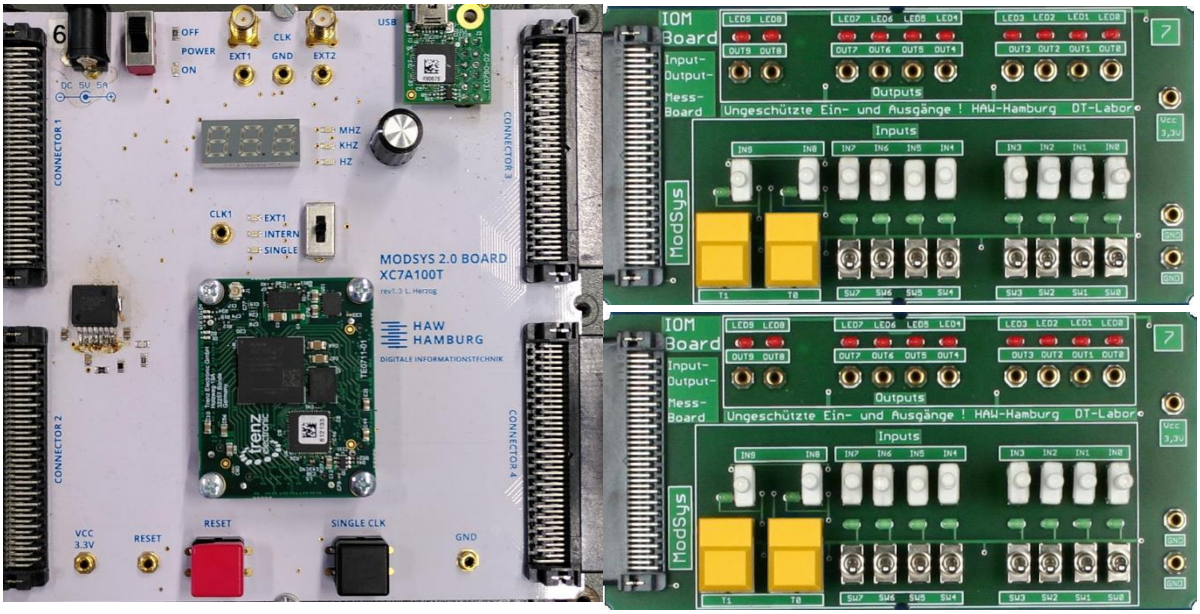


Fig. 2: MODSYS 2.0 connected to 2 IOM Boards (via connector 3 and connector 4)

PART 1 – Digital Adder

Preparation

P1.1 – Full adder cell

A full adder is a digital circuit that performs the addition of three binary bits: two significant bits (A,B) and an input carry (c_i). It produces two outputs: a sum bit (S) and a carry-out bit (c_o). The sum represents the least significant bit of the result, while the carry-out is passed to the next higher bit position in multi-bit binary addition.

The functional description of a 1 bit full adder is given in the following truth table

c_i	B	A	c_o	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Tab. 1: Full Adder Truth table

- Describe how the truth table can be transferred into Boolean equations
- Set up a corresponding VHDL code implementation of the Boolean equations

P1.2 – Ripple-Carry-Adder

- Download the VHDL sources `adder.vhd` (Design file) and `adder_tb.vhd` (Testbench) from TEAMs
- Familiarize yourself with both files:
  - How are the stimuli applied in the testbench?
  - What needs to be added for the –TODO sections?
  - …
- Add an implementation of an 8 bit ripple-carry-adder (RCA) to the adder process in the design section. Use the pre-given processing scheme.

Note:

The following expression allows parallelization of VHDL code within a process.

```
for i in 0 to 7 loop

-- VHDL code that is executed 8 times

end loop;
```

- Test your design by behavioral simulation means (EDAplayground or Vivado recommended, see appendix)
- Complement the TEST_DATA content by at least 5 more meaningful test vectors in the testbench

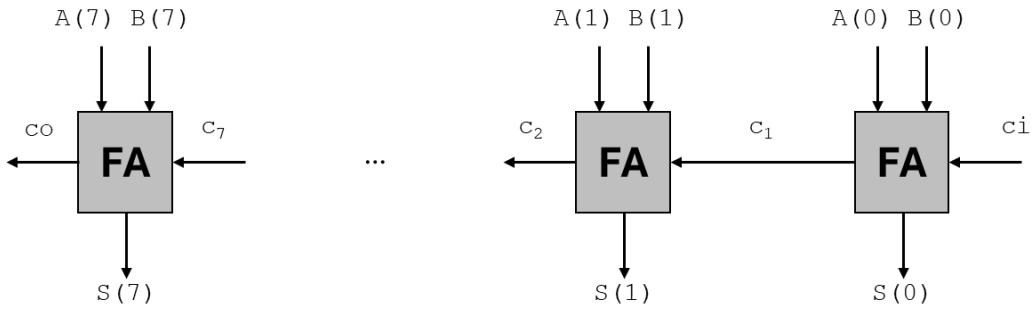


Fig. 3: The ripple-carry-adder (RCA) circuit performs `S = A + B + ci` with `A, B, S` as 8-bit vectors, `ci` as the carry input signal and `co` as the carry output signal.

P1.3 – Constraints Adder

- Download the VHDL sources `MODSYS2IOM.xdc`, from TEAMs
- Familiarize yourself with using the board resources (see also Fig. 4/5).
  - The switches are for the operands (one IOM at Connector 3 for `A`, IOM at Connector 4 for `B`)
  - The rightmost button (shown in yellow here) of the IOM board at Connector 3 is for the carry input `ci`.
  - The output LEDs of the IOM board at Connector 3 are used to show the sum `S` and the carry output `co`.
- Identify all constraints that must be set up/modified for mapping the adder as described above to the extension boards.

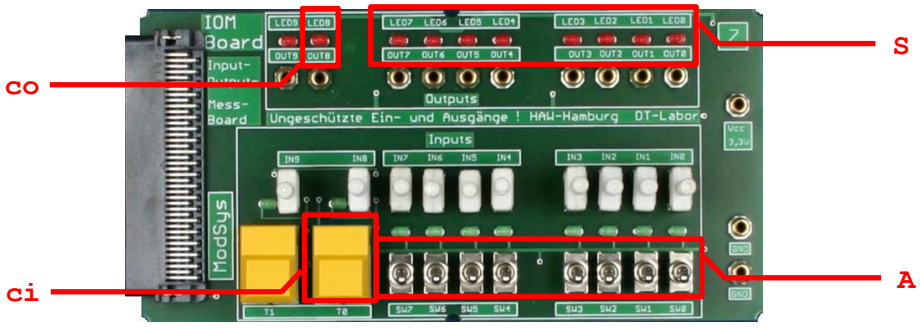


Fig. 4: PART 1 – VHDL Port mapping of Connector 3

Fig. 5: PART 1 - VHDL Port mapping of Connector 4

.

Task 1 – Adder

T1.1 – FPGA mapping Adder

- Bring a copy of your prepared files to the lab session, e.g. on an USB-Stick
- Create a new RTL project in Vivado (device XC7A100T CSG324-1)
- Add the prepared design and simulation (testbench) files as well as the constraint-file to the project.
- Perform the Implementation Design Flow. (no simulation, just generate Bitstream)
- Test the adder by applying your test vectors through the IOM-Boards

T1.2 – Timing analysis (also see Appendix)

- Logic synthesis results (Synthesis)
    - To open the results of the logical synthesis, click on SYNTHESIS->open synthesized design in the flow navigator
    - Navigate to the propagation delays, by clicking on Report Timing Summary (flow navigator) and ok afterwards. Next open the results (UNCONSTRAINED PATHS -> NONE to NONE -> SETUP) in the console.
    - Inspect given table entries of the given paths. Whate does the table show? What do the different PATH entries represent? Discuss the results.
- Physical Synthesis (Implementation)
    - Open the results of the physical synthesis, by clicking on Implementation->open implemented design in the flow navigator
    - Inspect the propagation delays, by clicking on Report Timing Summary (flow navigator). Next open the results (UNCONSTRAINED PATHS -> NONE to NONE -> SETUP).
    - Inspect given table entries of the given paths. Explain the difference compared to the results of the logic synthesis.

PART 2 – Subtractor Extension and status bits

Next, the given circuity has to be extended to carry calculate both additions and subtractions, which is achievable by transforming one operator into a negative number in advance. More precisely, the 2's complement is applied to operand `B` by bitwise negation and adding 1 to the LSB. The latter can be achieved by switching the input of `ci` to 1. To enable the circuit to execute both addition and subtraction, a multiplexer controlled by a select input (`sel`) has to be installed (see Fig. 6).

To identify specific properties of the actual calculation, a set of suitable status bits is typically generated. Here, the four conditions given in Tab. 2 are taken into account and the VHDL design needs to be extended accordingly.

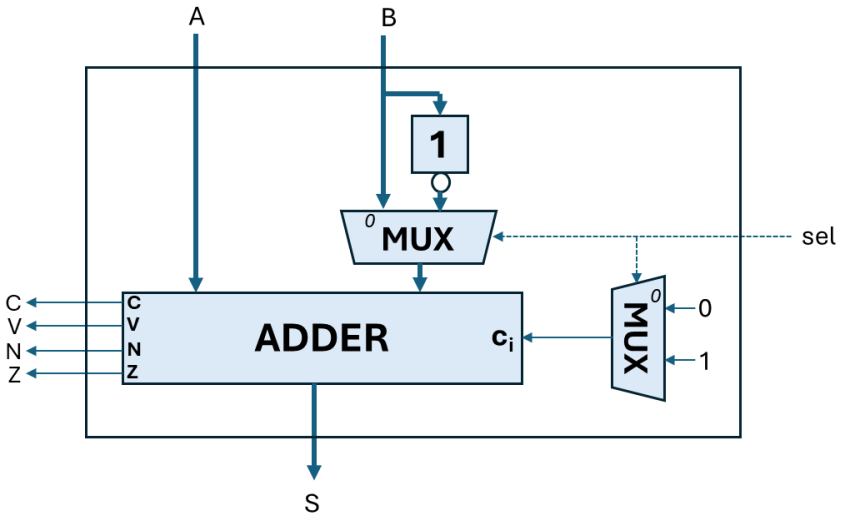


Fig. 6: Addsub module that switches between addition and subtraction depending on the selected input of the multiplexers (MUX): 0 for addition and 1 for subtraction. The outputs C, V, N, Z refer to the status flags Carry, Overflow, Negative and Zero, respectively.

Status flag	*Description*	*Formula*
C	Carry flag that denotes overflows of unsigned operations	C = co
V	Overflow flag that denotes overflows of signed operations	V = co ⊕ c_7
N	Negative flag that denotes negative results	N = S(7)
Z	Negative flag that denotes whether the result is zero	Z = 1, if S == 0 Z = 0, else

Tab. 2: Overview about the status flags

Preparation

P2.1 – Design file Addsub

- Make a copy of the existing adder design (`adder.vhd -> addsub.vhd`) and change the naming of the entity accordingly and download the VHDL source `addsub_tb.vhd` of the testbench from TEAMs

- Add the VHDL code for the Multiplexers given in Fig 6 to the process. Is the sensitivity list affected by this?
  Info: the c_7 needed for the calculation of the overflow flag, could be e.g. derived by inferring an extra variable storing the previous state of the carry flag.
- Add the status bit functionality given in Tab. 2 to process in the design file.
- Add the new ports to the entity of the design file and remove unnecessary ports (see Fig. 6).

P2.2 – Testbench Addsub

- Familiarize yourself with the new testbench `addsub_tb` module.
  - What has changed compared to the `adder_tb` and why?
- Set up at least 5 additional TEST_DATA stimuli vectors covering the operation of additions and subtractions as well as all the status flags.
- Perform the behavioral simulation to validate your design

P2.3 – Constraints Addsub

- Extend the constraint file by mapping the `sel` and status bits `C,V,N,Z` to the buttons and LEDs, respectively (see Fig. 7/8).

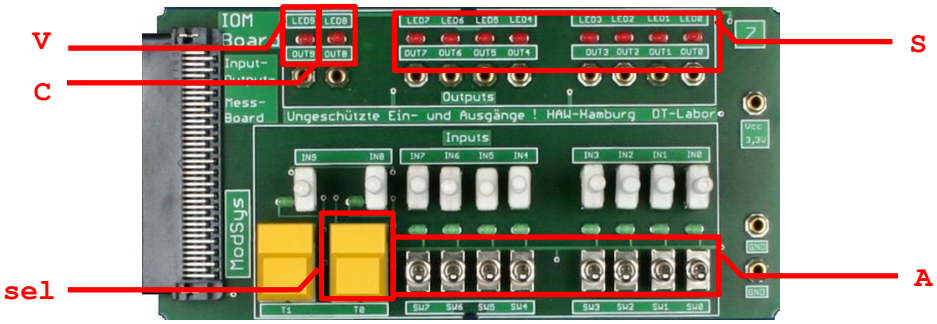


Fig. 7: PART 2 – VHDL Port mapping of Connector 3

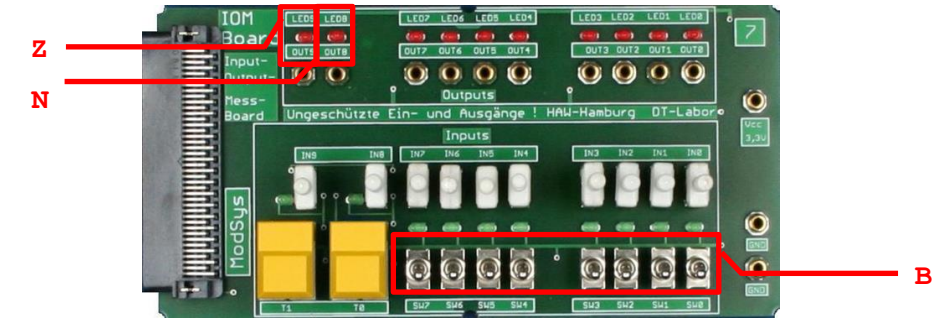


Fig. 8: PART 2 - VHDL Port mapping of Connector 4

Task 2 – Addsub

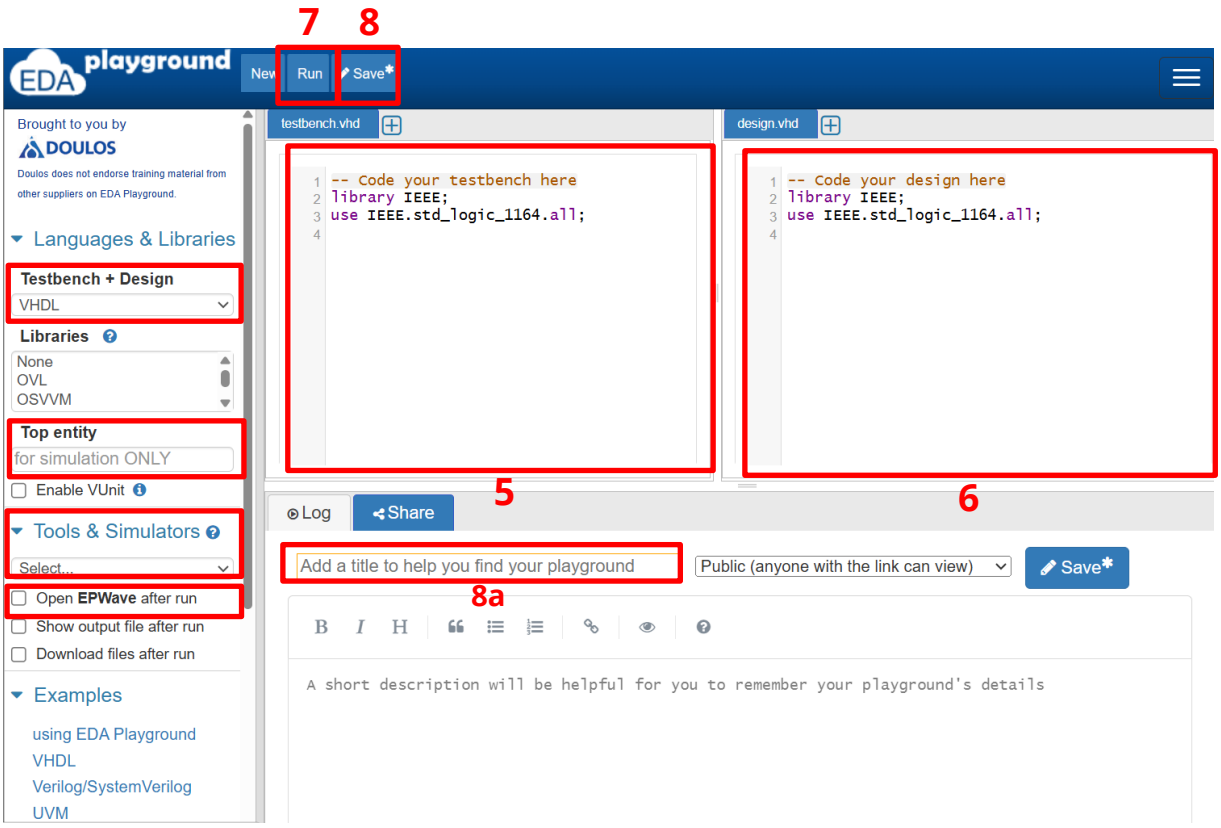
T2.1 – FPGA mapping Addsub

- Bring a copy of your prepared files to the lab session, e.g. on an USB-Stick
- Create a new RTL project in Vivado (device XC7A100T CSG324-1)
- Add the VHDL files and the xdc-file from preparation to the project.
- Perform the Implementation Design Flow. (no simulation, just generate bitstream)
- Test the adder by applying your test vectors through the IOM-Boards

T2.2 – Timing analysis

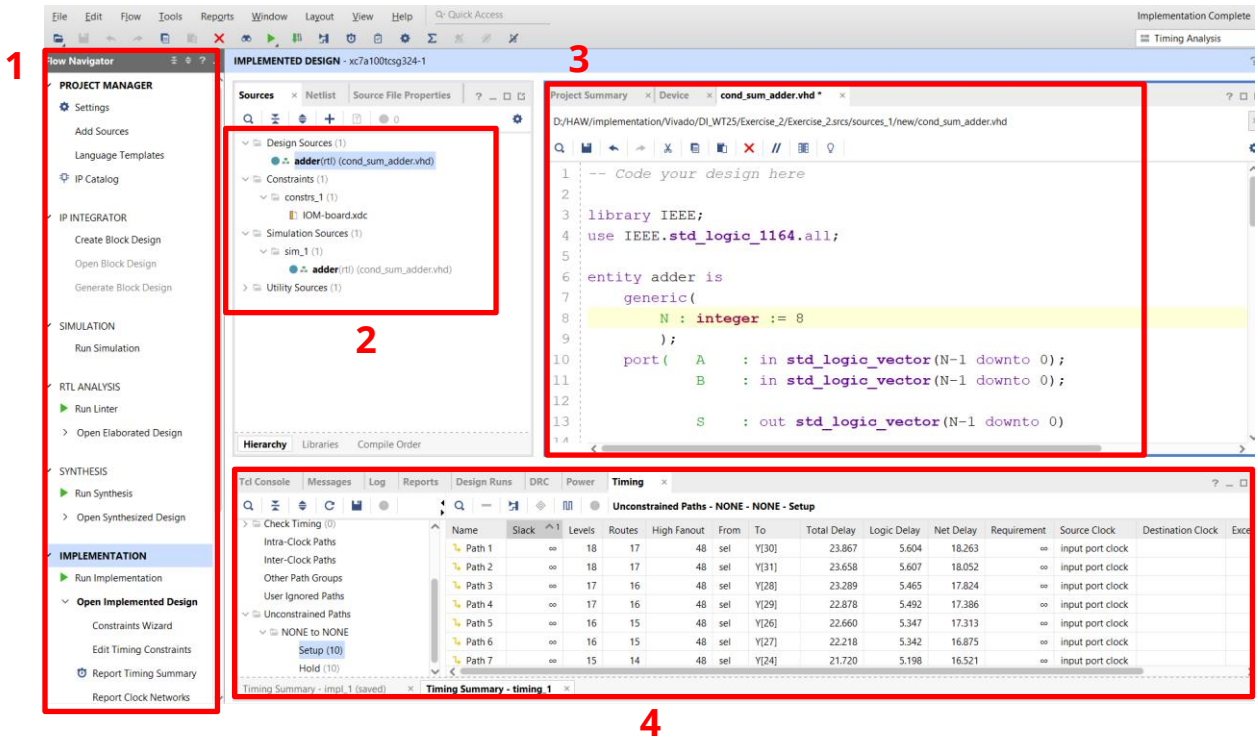
- Physical Synthesis (Implementation)
    - Open the results of the physical synthesis, by clicking on Implementation->open implemented design in the flow navigator
    - Inspect the propagation delays, by clicking on Report Timing Summary in the flow navigator and open the results: UNCONSTRAINED PATHS -> NONE to NONE -> SETUP).
    - Inspect given table entries of the given paths and compare the results to the results from T1.2.

Appendix –Overview EDAplayground



1. Set to VHDL
2. Insert the name of the testbench entity (not the filename)
3. Select simulator (e.g. Aldec Riviera Pro)
4. Enable to open the timing diagram after run
5. VHDL testbench code editor
6. VHDL design code editor
7. run project
8. Save project
    a. title of the project

Appendix –Overview Vivado



1. Flow navigator:
    a. For starting the synthesis steps, bitstream generation, etc.
    b. For opening designs and analyse timing
2. Design sources
    a. Design, cornstraints, simulation/testbench
3. Editor
    a. VHDL files, summary, etc
4. Console
    a. Design flow outputs (Logs)
    b. Design analysis outputs: Timing, Power, etc.